FOCUS FLOW

System Design Document

Version 1.0

27/02/2026

Table of Contents

Introduction

1.1. Purpose of the SDD

The purpose of this Software Design Document (SDD) is to provide a comprehensive overview of the design and architecture of **Focus Flow**, an AI-Proctored Examination and Learning Management System.

This document describes the system structure, architectural components, design decisions, and the interactions between different modules of the application.

It provides detailed technical insight into how Focus Flow implements secure exam delivery, integrated coding assessments, AI-based proctoring, and performance analytics.

This document serves as a reference for developers, project guides, testers, and stakeholders involved in the development and deployment of the system.

It ensures a shared understanding of the system's internal design and supports maintainability, scalability, and future enhancements.

General Overview and Design Guidelines / Approach

2.1 General Overview

Focus Flow is designed to provide a secure, unified, and intelligent platform for conducting online examinations and technical assessments.

The system integrates exam delivery, coding evaluation, and AI-based proctoring into a single web application to eliminate the need for multiple fragmented tools.

It aims to ensure academic integrity while maintaining user privacy through client-side processing of proctoring data.

The application follows a client-server architecture, consisting of a web-based frontend, a backend REST API server, and a centralized database system for data storage and retrieval.

The frontend provides an interactive and responsive interface for Students and Instructors to perform their respective activities.

The backend is responsible for authentication, authorization, exam management, question handling, result computation, and storage of proctoring flag events.

The system incorporates a built-in coding sandbox environment that enables students to write and execute code directly within the browser.

Additionally, the AI Proctoring Module operates locally on the client device to monitor suspicious behaviors such as gaze deviation and tab switching. Only generated flag events are transmitted to the backend server for review.

The system is designed to be scalable, secure, modular, and easy to maintain.

2.2 Assumptions/Constraints/Risks

2.2.1 Assumptions

• Users will have access to a modern web browser such as Chrome, Edge, or Firefox.

• Students attempting proctored examinations will have a functional webcam enabled on their device.

• Users will have a stable internet connection throughout the examination session.

• Educational institutions deploying the system will provide necessary infrastructure support for hosting and maintenance.

• The client device will have sufficient processing capability to execute the AI-based proctoring module locally.

2.2.2 Constraints

• The system must support a large number of concurrent users during peak examination periods.

• The AI proctoring module must operate entirely on the client side to preserve user privacy and reduce server load.

• The development timeline is constrained by academic project deadlines.

• The application must be compatible with modern web browsers and responsive across multiple device screen sizes.

• The system will be developed using the following technologies:
 o Frontend: React.js, Bootstrap 5
 o Backend: Node.js, Express.js
 o Database: MongoDB Atlas
 o AI Module: MediaPipe (Client-Side Processing)

• The system depends on the following external libraries and frameworks:
 o Passport.js: Authentication middleware for secure session handling
 o Monaco Editor: Integrated browser-based coding environment
 o MediaPipe: Machine learning framework for client-side proctoring

2.2.3 Risks

• High concurrent usage during examination periods may impact server performance and response time.

• Client-side AI processing may produce false positives in detecting suspicious behavior.

• Network interruptions during an active exam session may disrupt submissions or data synchronization.

• Browser compatibility issues may affect webcam access or coding sandbox functionality.

• Improper configuration of authentication mechanisms may expose security vulnerabilities.

3.Design Considerations

3.1 Goals and Guidelines

The design of the application is guided by the following goals and guidelines:

- User-Friendly Interface: The application should have an intuitive and easy-to-use interface for users to navigate through quizzes, submit answers, and view results.
- Scalability: The application should be designed to handle a large number of concurrent users without compromising performance.
- Security: The application should implement appropriate security measures to protect user information and prevent unauthorized access.
- Reliability: The application should be robust and resilient, ensuring minimal downtime and data loss.
- Performance: The application should be optimized for efficient data processing, minimizing response times and resource utilization.
- Modularity: The system should be designed with modular components to promote code reusability, maintainability, and extensibility.

4.System Architecture and Architecture Design

• The system architecture of Focus Flow follows a modular and component-based design to fulfil its primary responsibilities, including user authentication, exam management, AI-based proctoring, coding evaluation, data storage, and result generation.

• The system is decomposed into multiple components and subsystems, each responsible for a specific functional domain. These components interact through well-defined interfaces to ensure scalability, maintainability, and security.

• **User Interface Component:**
The User Interface (UI) component provides a responsive and interactive web interface for Students and Instructors. It manages user input, renders exam questions, displays coding interfaces, and presents analytics dashboards. The UI communicates with the backend through RESTful API calls.

• **Authentication and Authorization Component:**
This component manages secure user login, registration, session handling, and role-based access control. It ensures that only authorized users can access exam creation, exam participation, and administrative features.

• **Exam Management Component:**
The Exam Management component allows Instructors to create, modify, and manage examinations. It handles exam configurations such as timer settings, question selection, and approval workflows. It also coordinates with the Question Bank and Coding Sandbox components.

• **Question Bank Component:**
The Question Bank component stores and manages multiple-choice and coding questions. It supports categorization, difficulty tagging, and retrieval of questions for exam creation.

• **Coding Sandbox Component:**
The Coding Sandbox component integrates a browser-based code editor that allows Students to write and execute code during technical assessments. It supports syntax highlighting and controlled execution environments.

• **AI Proctoring Component:**
The AI Proctoring component operates locally on the client device using machine learning models. It monitors user behavior such as gaze direction and tab switching. Instead of transmitting video data, it generates and sends structured flag events to the backend server for integrity review.

• **Data Management Component:**
The Data Management component interacts with the database to store user accounts, exam data, responses, flag logs, and analytics records. It ensures persistent storage and maintains data integrity.

• **Result Generation Component:**
The Result Generation component evaluates student responses, calculates scores, compiles coding results, and generates analytics reports for both Students and Instructors.

4.1 Use cases

• **Register:**
This use case represents the process of user registration in Focus Flow. Users provide required details and create an account to access the system.

• **Login:**
Users can log in using their registered credentials. This use case enables authenticated access based on assigned roles such as Student or Instructor.

• **Logout:**
Users can securely log out of their accounts, terminating active sessions.

• **Reset Password:**
Users who forget their password can initiate the password reset process through secure verification.

• **Create Exam:**
Instructors can create new examinations by defining exam details such as title, duration, and question configuration.

• **Edit Exam:**
Instructors can modify existing exams before they are published or started.

• **Delete Exam:**
Instructors can delete exams that are no longer required.

• **Manage Question Bank:**
Instructors can add, edit, or delete questions within the question bank. Questions may include multiple-choice or coding-type formats.

• **Add Question to Exam:**
Instructors can select questions from the question bank or create new questions to include in an exam.

• **Start Exam:**
Instructors can initiate an exam session, allowing students to participate.

• **End Exam:**
Instructors can terminate an active exam session.

• **Attempt Exam:**
Students can participate in an active exam by answering MCQs and solving coding problems within the system.

• **Initialize Proctoring Session:**
When a student begins an exam, the AI Proctoring module initializes and begins monitoring activity locally.

• **Submit Answer:**
Students can submit answers for MCQ and coding questions during the exam session.

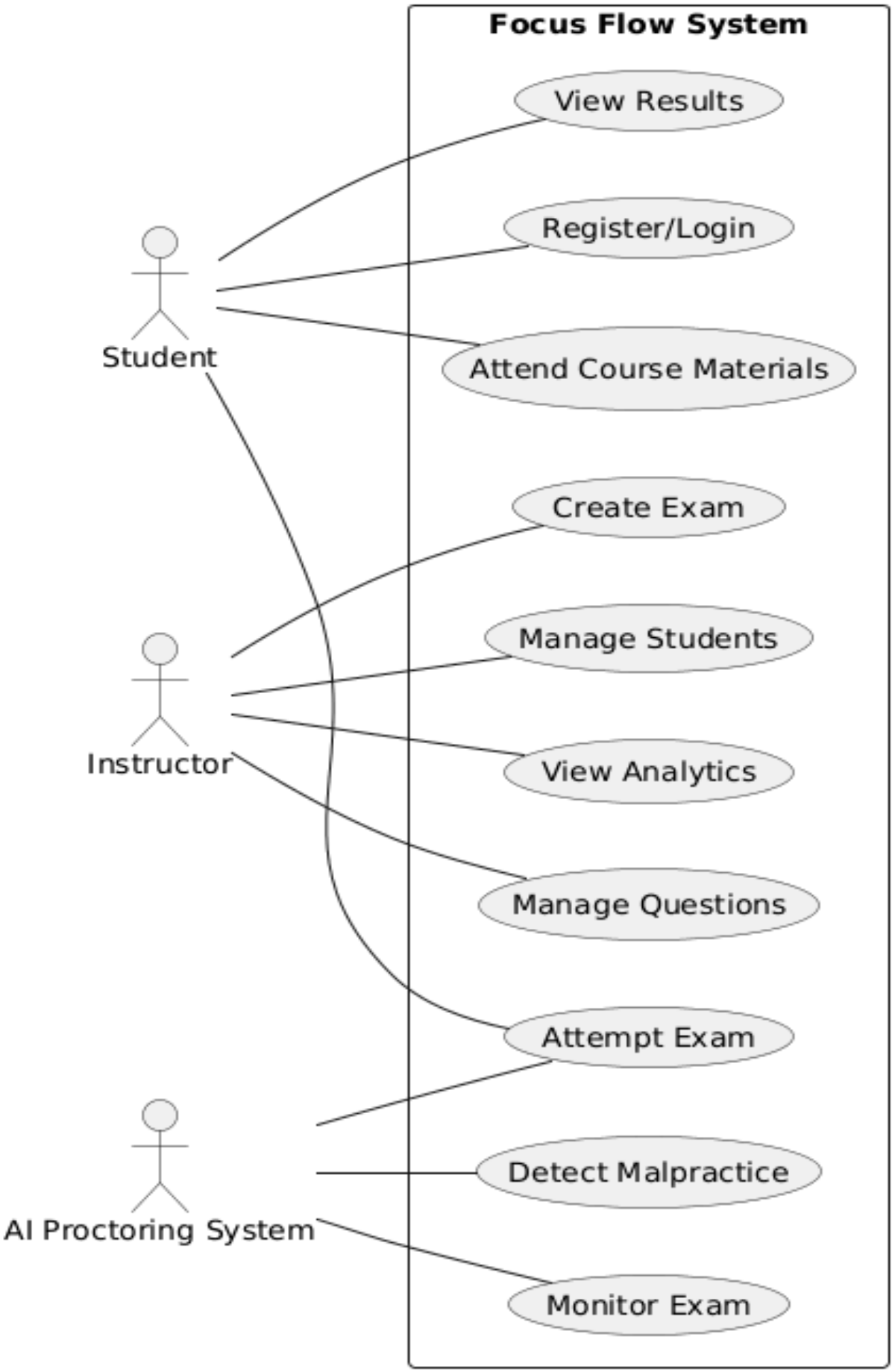
• **Generate Flag Event:**
The AI module generates flag events when suspicious behavior such as tab switching or gaze deviation is detected.

• **View Results:**
Students can view their scores and performance analytics after exam completion.

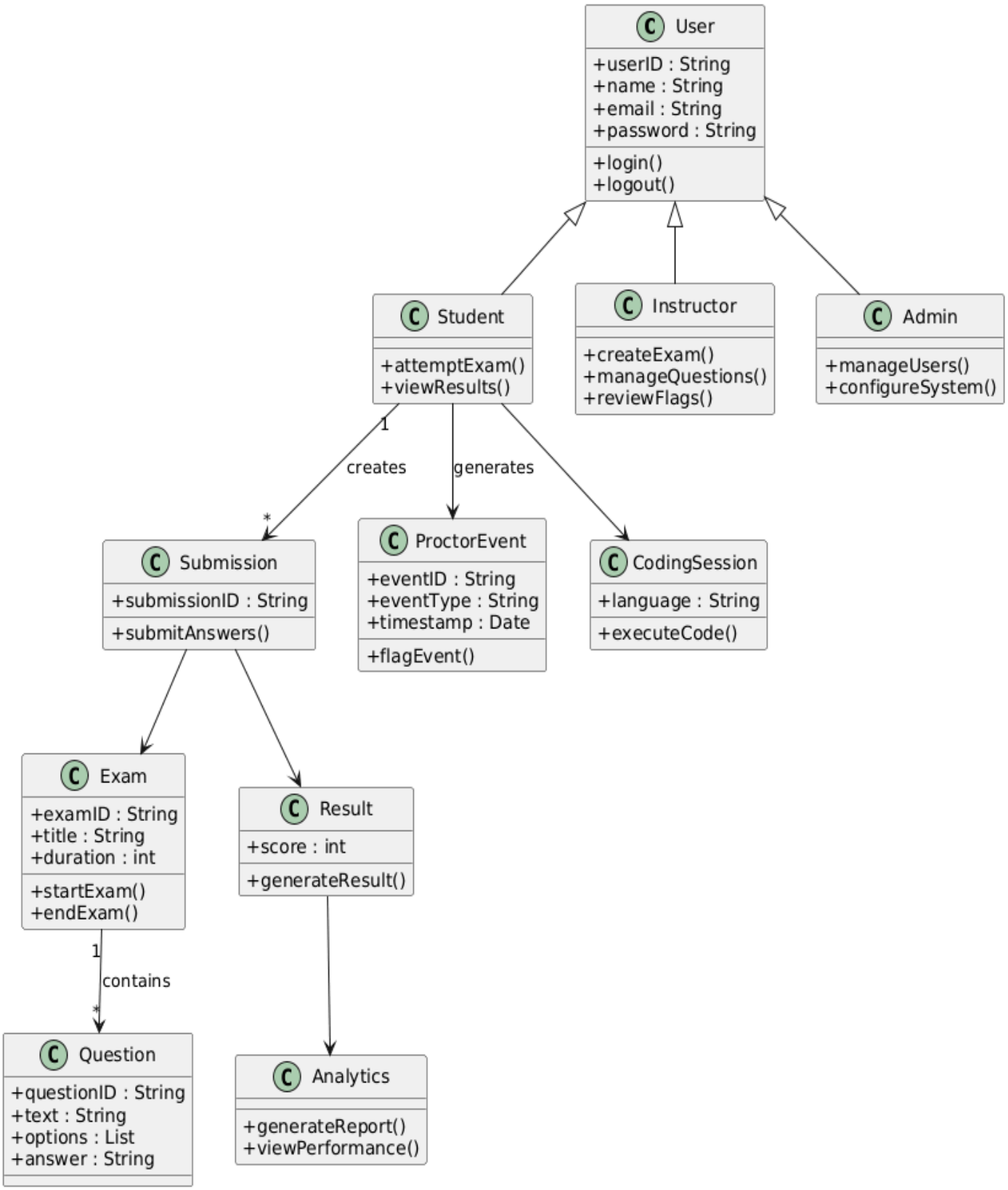
• **View Proctoring Logs:**
Instructors can review flag events and integrity logs associated with each exam attempt.

• **View Analytics Dashboard:**
Students and instructors can access performance insights and comparative analytics.

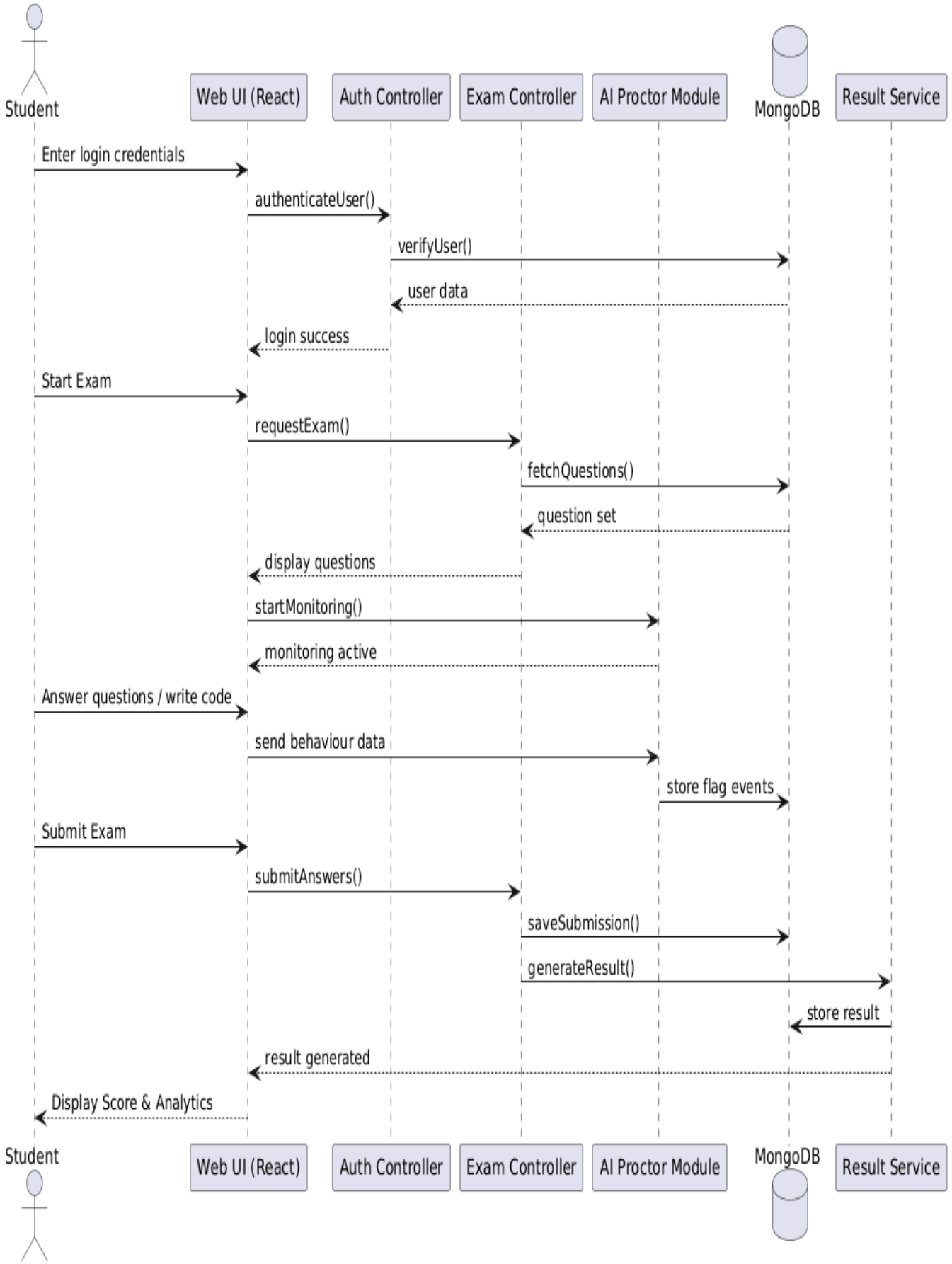
• **Manage Users:**
Authorized administrators can manage user roles and permissions within the system.


Focus Flow System
View Results
Register/Login
Attend Course Materials
Create Exam
Manage Students
View Analytics
Manage Questions
Attempt Exam
Detect Malpractice
Monitor Exam
Student
Instructor
AI Proctoring System

4.2 Class diagram



4.3 Sequence diagrams



4.4 Architectural Pattern

The Focus Flow application follows the **Model-View-Controller (MVC)** architectural pattern to ensure separation of concerns and maintain a modular system structure.

This architectural approach divides the application into three primary components:

• **Model:**
The Model represents the core data and business logic of the system. It includes entities such as Users, Exams, Questions, Responses, Flag Events, and Analytics records. The Model interacts directly with the MongoDB database to perform data storage and retrieval operations.

• **View:**
The View represents the frontend user interface developed using React.js and Bootstrap. It provides interactive dashboards for Students and Instructors, displays exam content, coding interfaces, analytics graphs, and proctoring status notifications.

• **Controller:**
The Controller manages user requests and handles communication between the View and the Model. Implemented using Node.js and Express.js, it processes API requests, enforces authentication rules, validates inputs, executes business logic, and returns appropriate responses to the frontend.

The MVC pattern enhances maintainability, scalability, and testability by ensuring that UI components, business logic, and data management remain independent and modular.

4.5 High-level architecture

• **Focus Flow Web Application:**
At the core of the architecture is the Focus Flow Web Application, which serves as the central platform for conducting secure online examinations. It integrates exam delivery, coding evaluation, AI-based proctoring, and analytics within a unified system.

• **User Interface Layer:**
The User Interface Layer is developed using React.js and provides separate dashboards for Students and Instructors. It handles user interactions such as exam participation, exam creation, viewing results, and managing question banks.

• **Authentication and Session Management:**
This component manages secure login, registration, and role-based access control using Passport.js. It ensures that only authorized users can access specific system functionalities based on their assigned roles.

• **Exam Management Module:**
The Exam Management module enables Instructors to create, configure, edit, and manage exams. It controls exam timers, question allocation, and participation permissions.

• **Question Bank Module:**
The Question Bank acts as a centralized repository of multiple-choice and coding questions. It supports categorization, difficulty levels, and efficient retrieval for exam creation.

• **Coding Sandbox Module:**
This module integrates a browser-based code editor environment that allows students to write and execute code during exams. It ensures controlled execution and secure submission handling.

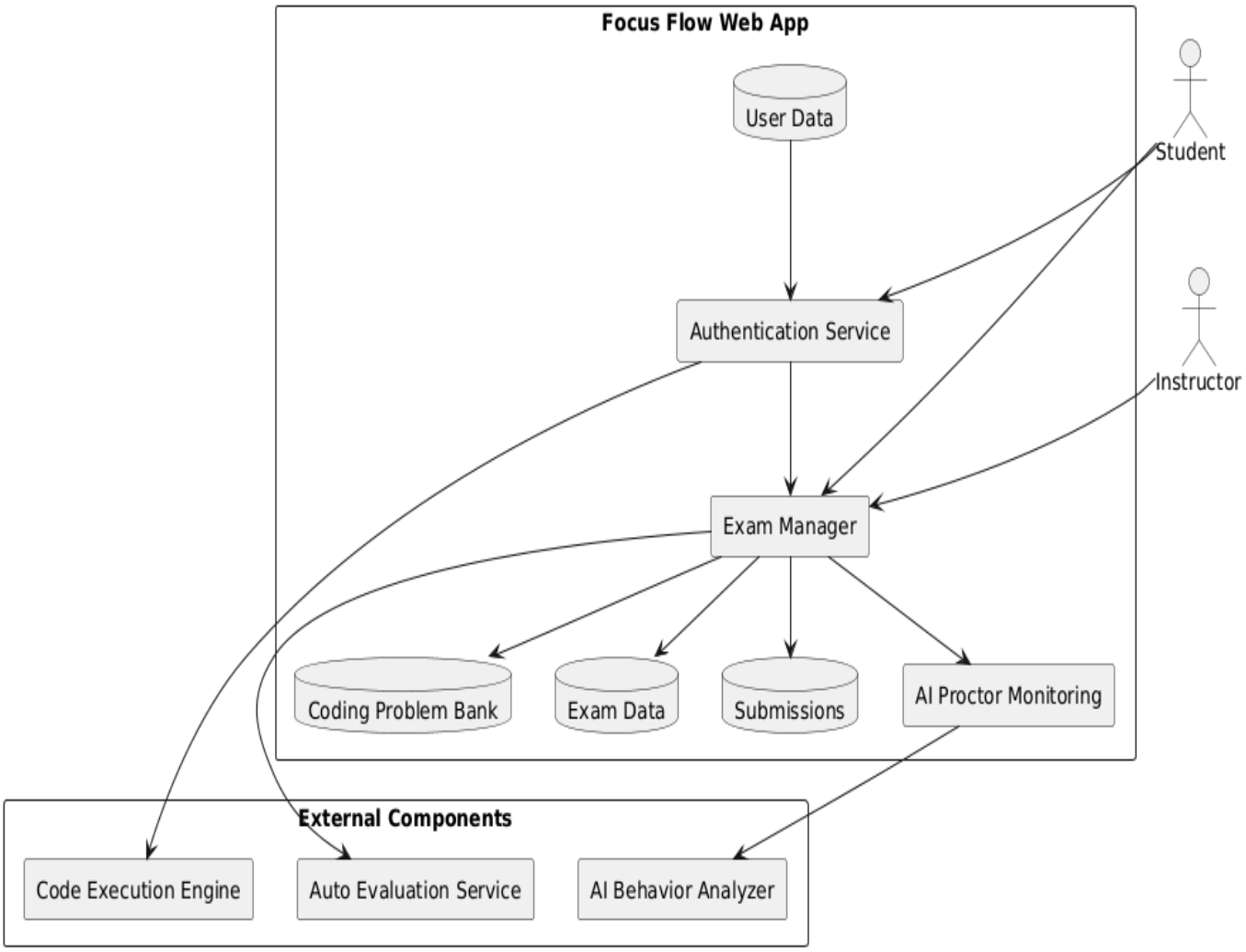
• **AI Proctoring Module:**
The AI Proctoring module operates locally on the client device using machine learning frameworks. It monitors behaviors such as gaze direction and tab switching. Instead of transmitting video streams, it generates structured flag events that are sent to the backend for logging and review.

• **Backend API Server:**
The Backend Server, built using Node.js and Express.js, processes RESTful API requests from the frontend. It handles authentication, exam logic, question management, response storage, and analytics generation.

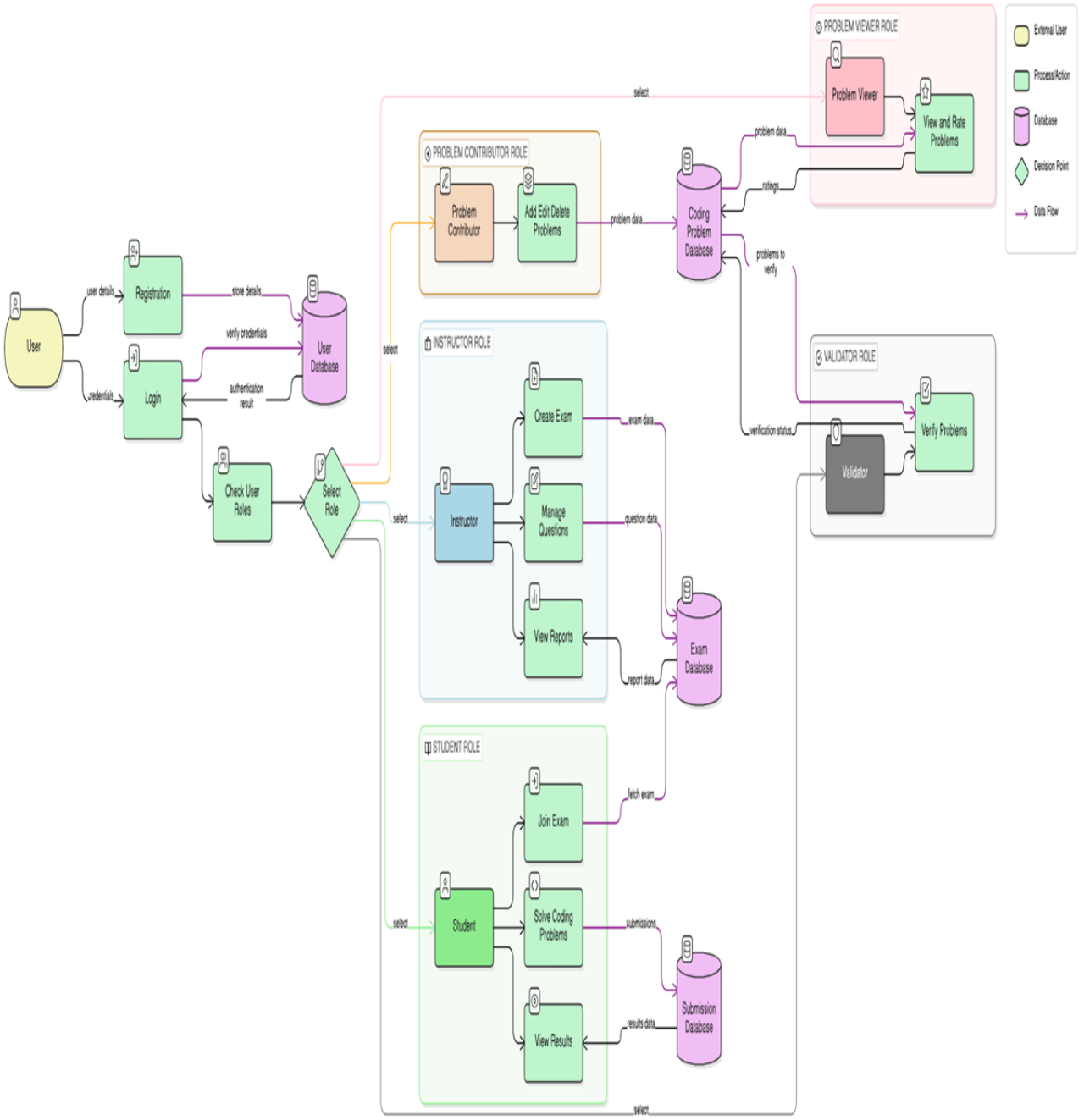
• **Database Layer:**
The Database Layer, implemented using MongoDB Atlas, stores persistent data including user credentials, exams, questions, responses, flag events, and analytics records.

• **External Stakeholders:**
The system interacts with Students, Instructors, and Educational Institutions who deploy and utilize the application. These external entities access the system through secure web interfaces.

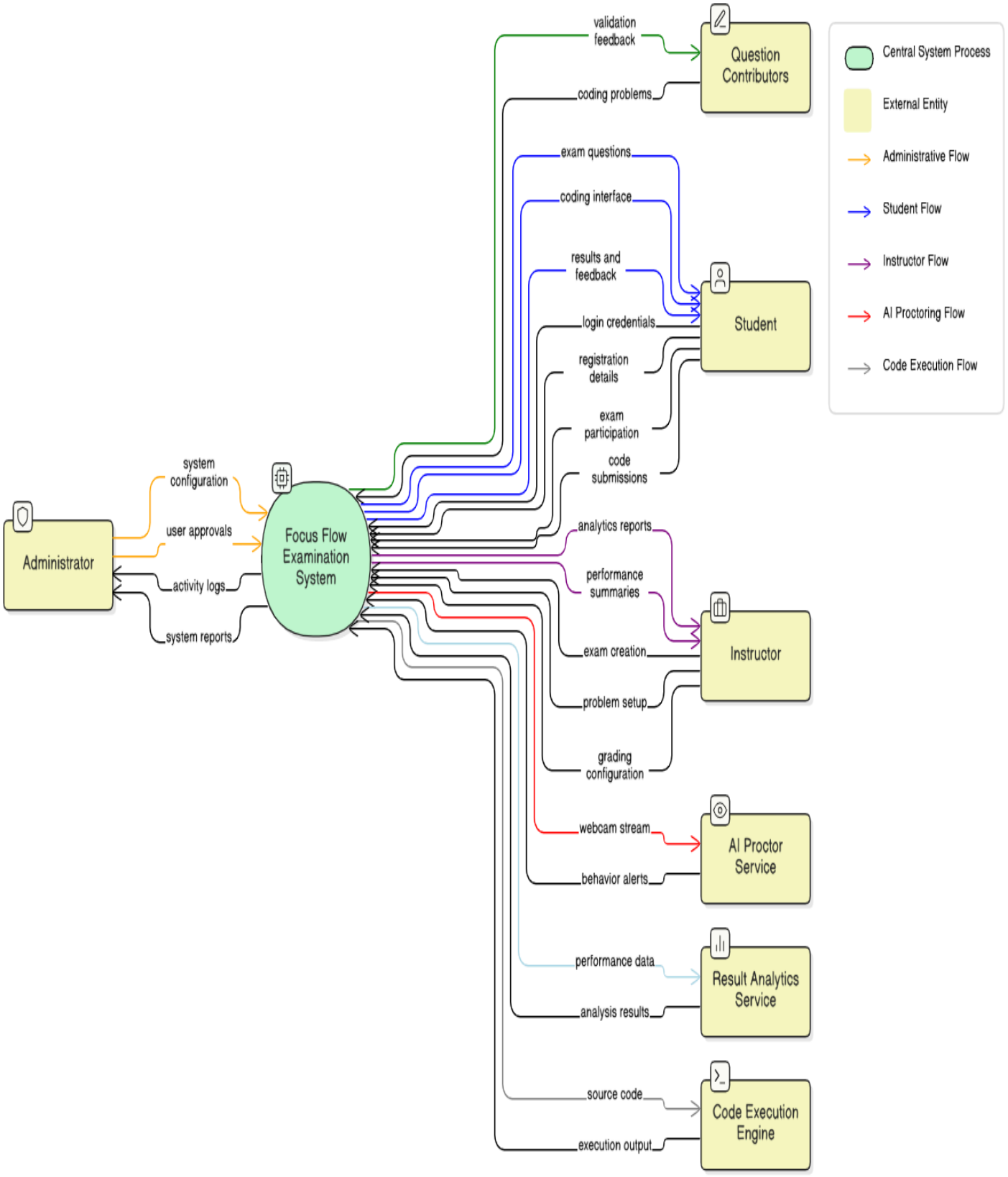
The High-Level Architecture ensures a clear separation between presentation, business logic, AI monitoring, and data storage, promoting scalability, reliability, and maintainability.



- Data Flow Diagram:

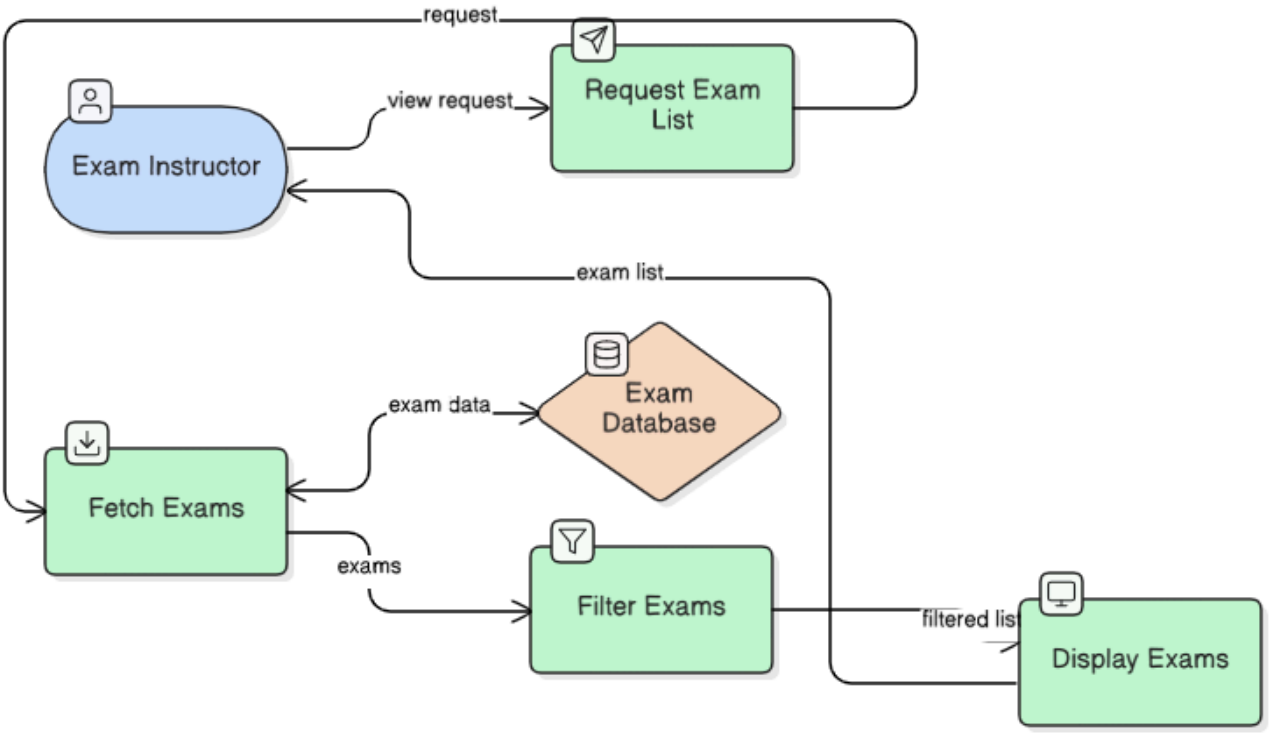


Level 0 DFD:


Administrator
Focus Flow Examination System
Question Contributors
Student
Instructor
AI Proctor Service
Result Analytics Service
Code Execution Engine
system configuration
user approvals
activity logs
system reports
validation feedback
coding problems
exam questions
coding interface
results and feedback
login credentials
registration details
exam participation
code submissions
analytics reports
performance summaries
exam creation
problem setup
grading configuration
webcam stream
behavior alerts
performance data
analysis results
source code
execution output
Central System Process
External Entity
Administrative Flow
Student Flow
Instructor Flow
AI Proctoring Flow
Code Execution Flow

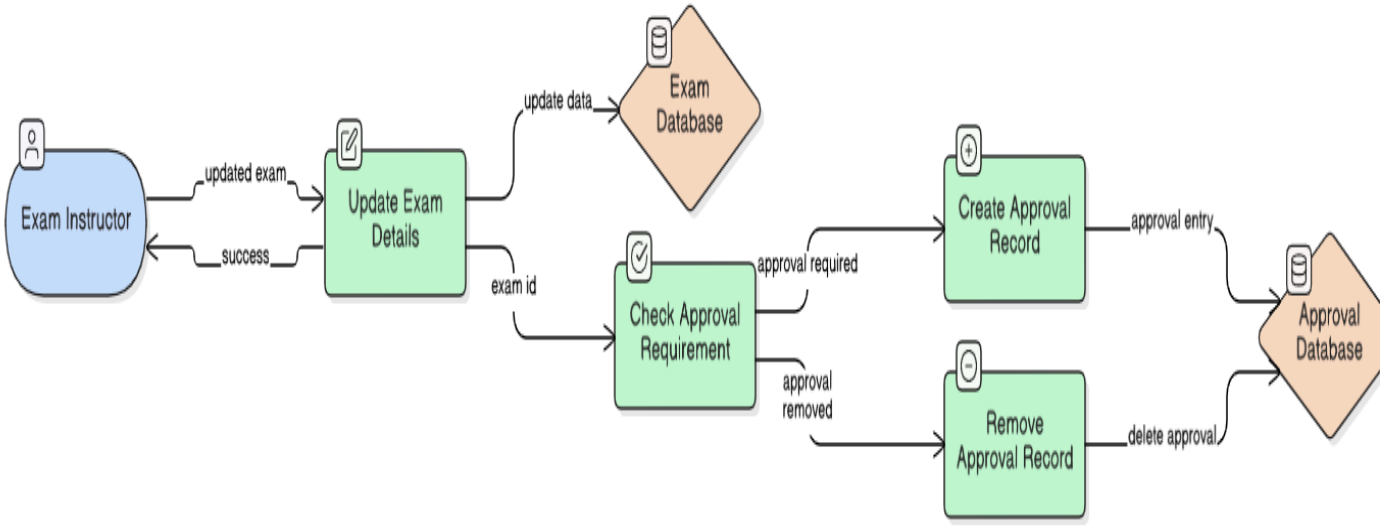
- Level 1 DFDs:

Create Exam

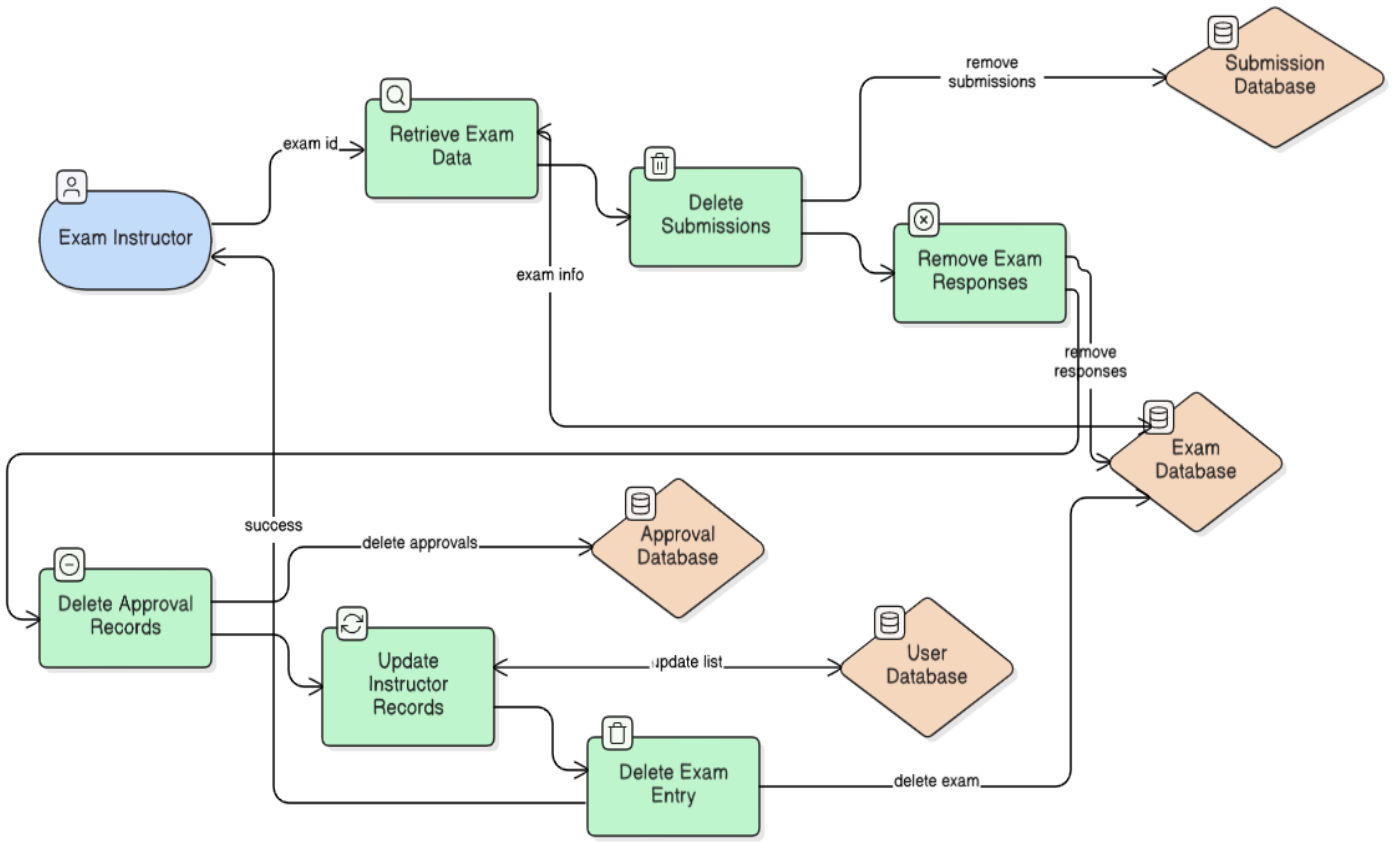


View Exams

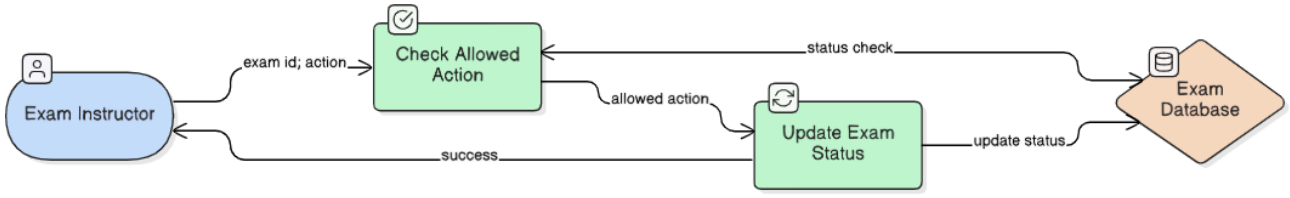
Edit Exam



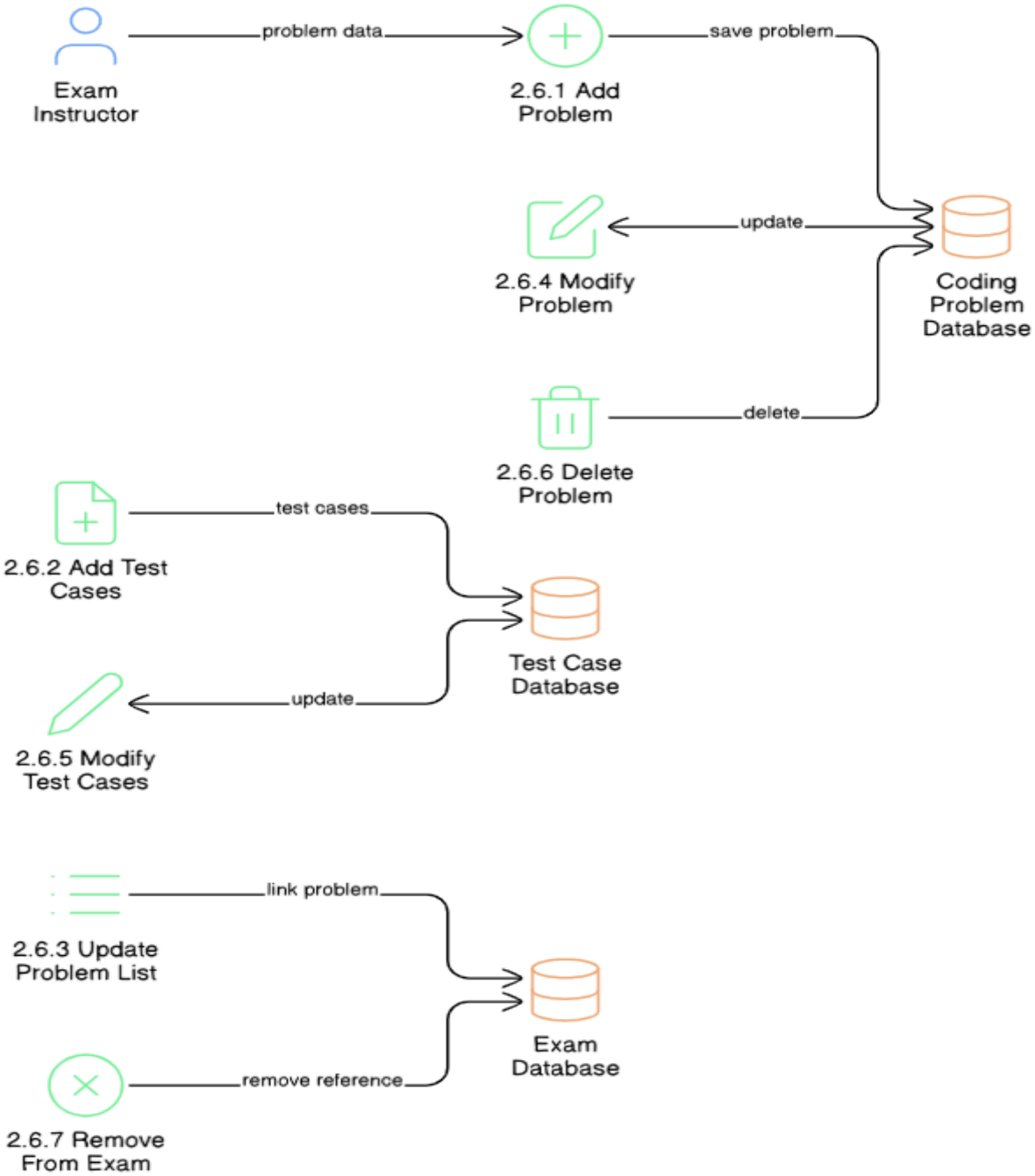
Delete Exam



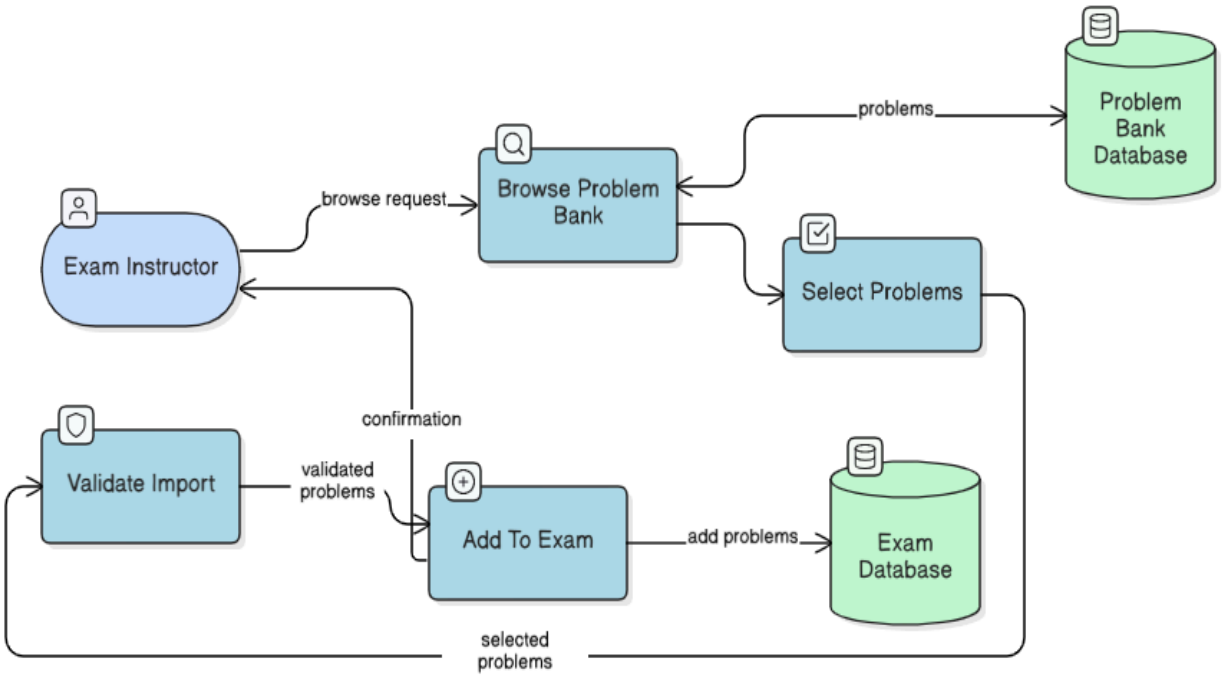
Start, End Exam



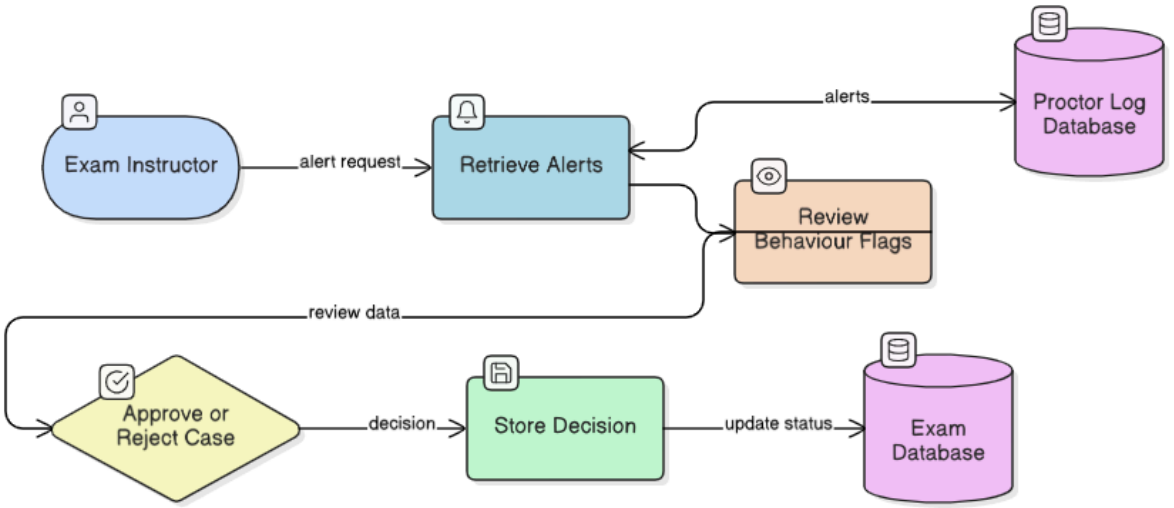
Manage Coding Problems



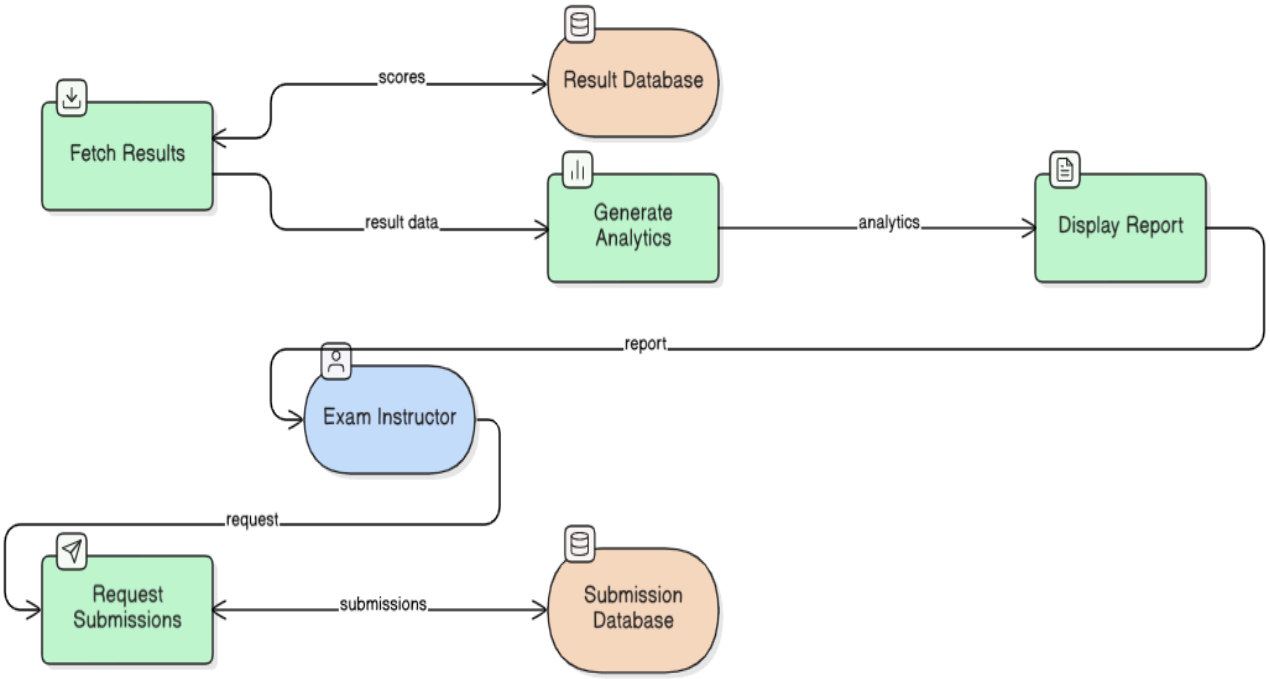
Import From Problem Bank



Manage Proctor Flags / Approval



View Submissions And Results



4.6 Technology stack

• **Operating System:**
The Focus Flow application can operate on any modern operating system such as Windows, Linux, or macOS. Since it is a web-based application, it is platform-independent and accessible through standard web browsers.

• **Programming Language:**
The primary programming language used in the development of the system is JavaScript.

• **Frontend Technologies:**
o React.js: Used for building dynamic and responsive user interfaces.
o Bootstrap 5: Used for responsive design and consistent UI styling.
o MediaPipe: Used for implementing client-side AI-based proctoring functionalities.
o Monaco Editor: Integrated for browser-based coding sandbox support.

• **Backend Technologies:**
o Node.js: Provides the runtime environment for server-side execution.
o Express.js: Used for building RESTful APIs and managing routing logic.
o Passport.js: Used for secure authentication and session management.

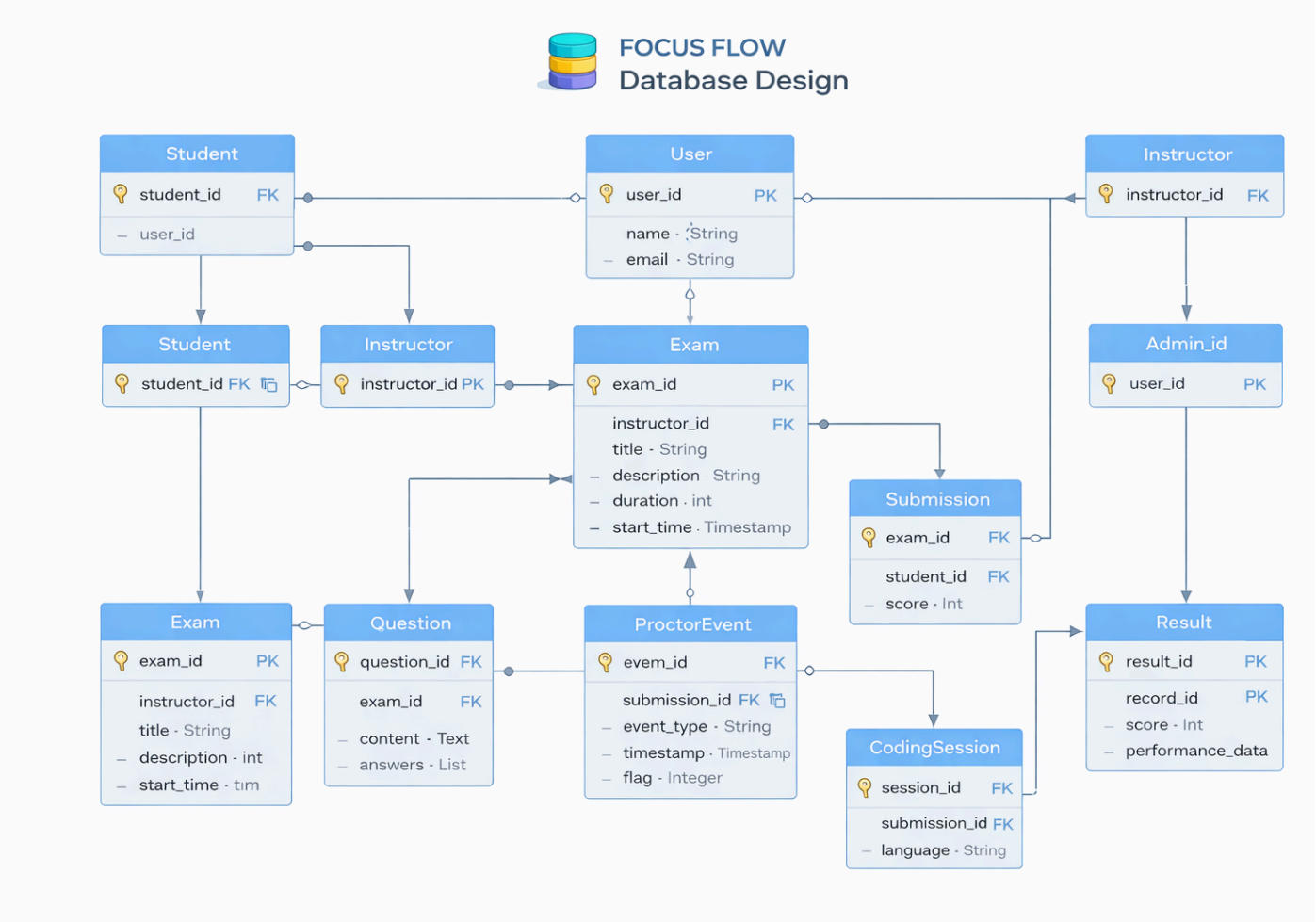
• **Database:**
o MongoDB Atlas: A cloud-based NoSQL database used for storing user data, exams, responses, flag events, and analytics information.

• **Web Server:**
The backend server is hosted using Node.js with Express middleware to handle HTTP requests and responses efficiently.

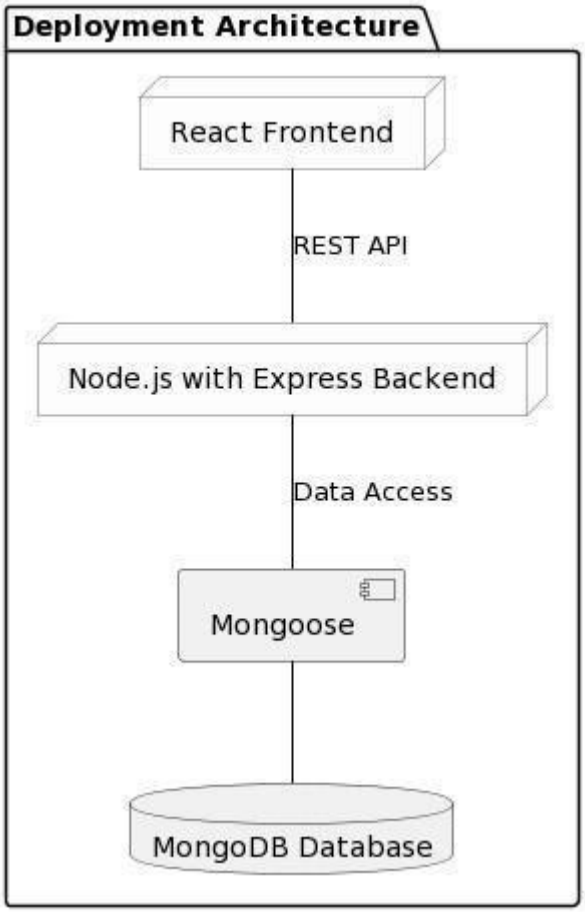
• **Deployment Environment:**
The application can be deployed on cloud platforms such as Render, Vercel, or similar hosting services that support Node.js applications and MongoDB connectivity.

The chosen technology stack ensures scalability, modularity, and efficient performance for both academic deployment and potential production-level expansion.

4.7 Database Design



4.8 Deployment Architecture



5.API Documentation

User registration, Login, User Management & Details

1. POST /register

Data to be sent in the body:

- username (string): The unique username chosen by the user.
- name (string): The full name of the user.
- email (string): The email address of the user.

- password (string): The password entered by the user.
- role (string): The role assigned to the user. Can be "student" or "instructor".

Responses:

- 400 Bad Request: Missing required parameters.
- 409 Conflict: Username or email already exists.
- 201 Created: User {username} registered successfully.
- 500 Internal Server Error: An error occurred while processing the request.

---

2. POST /auth

Data to be sent in the body:

- username (string): The registered username.
- password (string): The user's password.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Invalid credentials.
- 200 OK: Authentication successful. Access token and user role returned.

---

3. GET /refresh

Data to be sent in cookies:

- jwt (string): The refresh token stored in HTTP-only cookies.

Responses:

- 401 Unauthorized: Token not found.
- 403 Forbidden: Invalid or expired token.
- 200 OK: New access token generated successfully.

---

4. GET /logout

Data to be sent in cookies:

- jwt (string): The session token.

Responses:

- 204 No Content: User session not found.
- 200 OK: Successfully logged out.

5. POST /requestotp

Data to be sent in the body:

- email (string): The registered email address of the user.
- purpose (string): The purpose of OTP request. Can be "verification" or "reset".

Responses:

- 400 Bad Request: Missing required parameters.
- 404 Not Found: User not found.
- 409 Conflict: OTP already requested. Please wait before requesting again.
- 201 Created: OTP sent successfully to registered email.
- 500 Internal Server Error: Error while sending OTP.

---

6. POST /resetpassword

Data to be sent in the body:

- email (string): The registered email address.
- otp (string): The OTP sent to the user's email.
- newpassword (string): The new password to be set.

Responses:

- 400 Bad Request: Missing required parameters.
- 401 Unauthorized: Invalid OTP or expired OTP.
- 404 Not Found: User not found.
- 200 OK: Password reset successfully.

---

7. POST /verify

Data to be sent in the body:

- email (string): The registered email address.
- otp (string): The OTP received for account verification.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Invalid OTP.
- 200 OK: Account verified successfully.

8. GET /roles

Responses:

- 200 OK: Returns the role of the authenticated user.
- 401 Unauthorized: User not authenticated.

Exam Management

---

1. POST /exam/create

Data to be sent in the body:

- title (string): The name of the exam.
- description (string): Short description of the exam.
- duration (number): Duration of the exam in minutes.
- approvalrequired (boolean): Whether the exam requires approval before participation.

Responses:

- 400 Bad Request: Missing required parameters.
- 401 Unauthorized: User not authorized to create exams.
- 201 Created: Exam created successfully with exam ID {examId}.
- 500 Internal Server Error: Error occurred while creating exam.

---

2. PUT /exam/edit

Data to be sent in the body:

- examid (string, required): The ID of the exam.
- title (string, optional): Updated exam title.
- description (string, optional): Updated description.
- duration (number, optional): Updated duration.

Responses:

- 400 Bad Request: Missing or invalid parameters.
- 401 Unauthorized: User not authorized to edit this exam.
- 404 Not Found: Exam not found.
- 200 OK: Exam updated successfully.

---

3. POST /exam/delete

Data to be sent in the body:

- examid (string, required): The ID of the exam to delete.

Responses:

- 400 Bad Request: Missing exam ID.
- 401 Unauthorized: User not authorized to delete this exam.
- 404 Not Found: Exam not found.
- 200 OK: Exam deleted successfully.

---

4. PUT /exam/start

Data to be sent in the body:

- examid (string, required): The ID of the exam to start.

Responses:

- 400 Bad Request: Missing exam ID.
- 401 Unauthorized: User not authorized to start this exam.
- 404 Not Found: Exam not found.
- 409 Conflict: Exam already started.
- 200 OK: Exam started successfully.

---

5. PUT /exam/end

Data to be sent in the body:

- examid (string, required): The ID of the exam to end.

Responses:

- 400 Bad Request: Missing exam ID.
- 401 Unauthorized: User not authorized to end this exam.
- 404 Not Found: Exam not found.

- 409 Conflict: Exam already ended.
- 200 OK: Exam ended successfully.

---

6. POST /exam/info

Data to be sent in the body:

- pagenumber (number, required)
- pagesize (number, required)
- filter (string): Type of exams to retrieve. Can be "created", "attempted", or "pending".

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Authentication failed.
- 200 OK: Returns list of exams.

---

7. POST /exam/requestapproval

Data to be sent in the body:

- examid (string, required): The ID of the exam.

Responses:

- 400 Bad Request: Missing exam ID.
- 401 Unauthorized: Not allowed to request approval.
- 404 Not Found: Exam not found.
- 409 Conflict: Approval already requested.
- 200 OK: Approval request sent successfully.

Question Bank Management

---

1. POST /question/add

Data to be sent in the body:

- type (string, required): Type of question. Can be "mcq" or "coding".
- question (string, required): The question text.
- options (array of strings, required for mcq): Must contain at least 4 options.
- answer (string, required): Correct answer (for mcq) or expected output/test case (for coding).
- difficulty (string, required): Can be "easy", "medium", or "hard".
- category (string, required): Category of the question.
- subcategory (string, optional): Subcategory of the question.

Responses:

- 400 Bad Request: Missing or invalid parameters.
- 401 Unauthorized: User not authorized to add questions.
- 201 Created: Question added successfully.
- 500 Internal Server Error: Error occurred while saving question.

---

2. PUT /question/edit

Data to be sent in the body:

- questionid (string, required): ID of the question.
- question (string, optional): Updated question text.
- options (array of strings, optional): Updated options.
- answer (string, optional): Updated answer.
- difficulty (string, optional): Updated difficulty level.
- category (string, optional): Updated category.

(At least one property must be provided.)

Responses:

- 400 Bad Request: Invalid update request.
- 401 Unauthorized: User not authorized to edit this question.
- 404 Not Found: Question not found.
- 200 OK: Question updated successfully.

---

3. POST /question/delete

Data to be sent in the body:

- questionid (string, required): ID of the question.

Responses:

- 400 Bad Request: Missing question ID.
- 401 Unauthorized: User not authorized to delete this question.
- 404 Not Found: Question not found.
- 200 OK: Question deleted successfully.

---

4. POST /question/list

Data to be sent in the body:

- pagenumber (number, required)
- pagesize (number, required)
- category (string, optional)
- difficulty (string, optional)
- type (string, optional): "mcq" or "coding"

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Authentication failed.
- 200 OK: Returns list of questions.

---

5. POST /question/addtoexam

Data to be sent in the body:

- examid (string, required): The ID of the exam.
- questionid (string, required): The ID of the question.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Not authorized to modify this exam.

- 404 Not Found: Exam or question not found.
- 200 OK: Question added to exam successfully.

Exam Participation & Submission

---

1. PUT /exam/session/initialize

Data to be sent in the body:

- examid (string, required): The ID of the exam to initialize.

Responses:

- 400 Bad Request: Missing exam ID.
- 401 Unauthorized: User not authorized to participate.
- 403 Forbidden: Exam not started or already completed.
- 404 Not Found: Exam not found.
- 200 OK: Exam session initialized successfully.

---

2. POST /exam/questions

Data to be sent in the body:

- examid (string, required): The ID of the exam.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Authentication failed.
- 403 Forbidden: Exam not active.
- 404 Not Found: Exam not found.
- 200 OK: Returns exam questions and initializes response record.

---

3. PUT /exam/submitanswer

Data to be sent in the body:

- examid (string, required): The ID of the exam.
- questionid (string, required): The ID of the question.
- answer (string, required): Selected answer (for MCQ) or code submission (for coding question).

Responses:

- 400 Bad Request: Missing parameters or question already answered.
- 401 Unauthorized: Authentication failed.
- 403 Forbidden: Exam ended or not started.
- 404 Not Found: Exam or question not found.
- 201 Created: Answer submitted successfully.

---

4. POST /exam/submitfinal

Data to be sent in the body:

- examid (string, required): The ID of the exam.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Authentication failed.
- 403 Forbidden: Exam not active.
- 404 Not Found: Exam not found.
- 200 OK: Exam submitted successfully. Result generated.

AI Proctoring & Flag Management

---

5. POST /proctor/flag

Data to be sent in the body:

- examid (string, required): The ID of the exam.
- eventtype (string, required): Type of violation. Can be "gaze", "tabswitch", or "multiplefaces".
- timestamp (string, required): Time of detected event.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Authentication failed.
- 404 Not Found: Exam not found.
- 201 Created: Flag event recorded successfully.

---

6. POST /proctor/logs

Data to be sent in the body:

- examid (string, required): The ID of the exam.
- student (string, optional): Username of the student.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Not authorized to view logs.
- 404 Not Found: No logs found.
- 200 OK: Returns list of flag events.

Result & Analytics Management

---

7. POST /result/view

Data to be sent in the body:

- examid (string, required): The ID of the exam.

Responses:

- 400 Bad Request: Missing parameters.
- 401 Unauthorized: Authentication failed.
- 404 Not Found: Result not found.
- 200 OK: Returns exam score and performance summary.

8. GET /analytics/dashboard

Responses:

- 401 Unauthorized: Authentication failed.
- 200 OK: Returns performance statistics and comparative analytics data.

Appendix A: Record of Changes

Version Number	Date	Author/Owner	Description of Change
<0.1>	<25/02/2026>	Team 14	Increased the no. of stakeholders from 3 to 5 to fit the app requirements
<1.0>	<27/02/2026>	Team 14	Increased the no. of stakeholders from 5 to 6 to fit the app requirements

Appendix B: Acronyms

Acronym	Literal Translation
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets